

Dhirubhai Ambani Institute of Information & Communication Technology
End Semester Examination — Summer Semester 2024-2025

Course Title: **IT215 — System Programming**

Max Marks: **74**

Date: _____

Time: **120 minutes**

Student Name: _____

Student ID: _____

Instructions:

- All questions are compulsory.
- Write your answers in the space provided after each question.
- Write concisely and clearly. Long answers do not fetch higher scores.
- Assume all system calls execute without errors unless stated otherwise.
- Assume each call to printf flushes its output just before it returns.

Q1 Concurrent Programming and Process Creation [16 Marks]

Consider code1, code2, code3, and code4 below. For each code snippet, list **all possible values** that the variable **result** can take when it is printed **for the first time** on stdout. Briefly justify each answer. (4 marks each)

// code1 — threads, no synchronisation

```
int result = 0;
void worker() {
    result += 3;
    printf("%d\n", result);
}
int main() {
    pthread_t t[2]; int i;
    for (i = 0; i < 2; i++)
        pthread_create(&t[i], NULL, (void*)worker, NULL);
    result += 3;
    printf("%d\n", result);
}
```

// code2 — threads with pthread_join after each create

```
int result = 0;
void worker() {
    result += 3;
    printf("%d\n", result);
}
int main() {
    pthread_t t[2]; int i;
    for (i = 0; i < 2; i++) {
        pthread_create(&t[i], NULL, (void*)worker, NULL);
        pthread_join(t[i], NULL);
    }
    result += 3;
    printf("%d\n", result);
}
```

// code3 — threads with mutex lock

```
int result = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
```

```

void worker() {
    pthread_mutex_lock(&m);
    result += 3;
    printf("%d\n", result);
    pthread_mutex_unlock(&m);
}

int main() {
    pthread_t t[2]; int i;
    for (i = 0; i < 2; i++)
        pthread_create(&t[i], NULL, (void*)worker, NULL);
    result += 3;
    printf("%d\n", result);
}

```

// code4 — using fork() instead of threads

```

int result = 0;
void worker() {
    result += 3;
    printf("%d\n", result);
}

int main() {
    int i;
    for (i = 0; i < 2; i++) {
        if (fork() == 0) {
            worker();
            exit(0);
        }
    }
    result += 3;
    printf("%d\n", result);
}

```

Answer for Code1:

Answer for Code2:

Answer for Code3:

Answer for Code4:

Q2 Synchronisation with Concurrent Programming

a. The following function is called by exactly two threads concurrently. Identify the synchronisation problem present and rewrite the code to fix it. [2 Marks]

```
int flag = 0;
void* critical_task(void* arg) {
    if (flag == 0) {
        flag = 1;
    } else {
        return NULL; /* only one thread should enter */
    }
    /* === critical section === */
    flag = 0;
    return NULL;
}
```

Answer:

b. For each situation below, state the single most appropriate synchronisation primitive from: *none needed*, *mutex*, *semaphore(n)*, *rwlock*. If semaphore, specify initial value. Choose the simplest primitive that provides maximum concurrency. [6 Marks]

Situation	Description	Primitive
A.	Three threads only read a shared configuration struct; no thread ever writes to it.	
B.	Three threads each increment a shared integer counter.	
C.	One hundred threads search a read-only hash table; no modifications occur.	
D.	One hundred threads search a linked list; one thread periodically inserts new nodes.	
E.	At most three threads may execute a licensed feature simultaneously (licence limit = 3).	
F.	A logger thread waits for log events posted by worker threads; no event may be lost.	
G.	A kernel maintains an inode table readable by many threads but writable only by the VFS thread.	
H.	A system has three printer queues shared among many threads; each printer handles one job at a time.	
I.	A producer places items into a bounded circular buffer of size 50; it must block when the buffer is full.	
J.	A consumer removes items from the same size-50 circular buffer; it must block when the buffer is empty.	

c. The following partial code implements a bounded-buffer producer-consumer system with buffer size 64. There are multiple producer threads and multiple consumer threads. Fill in all the blanks to

make it correct and race-free. [9 Marks]

```
#include <pthread.h>
#include <semaphore.h>

#define BUF_SIZE 64
int buf[BUF_SIZE];
int in = 0, out = 0;
sem_t empty_slots, filled_slots;
pthread_mutex_t mutex;

void init() {
    sem_init( _____, 0, _____ ); /* empty_slots */
    sem_init( _____, 0, _____ ); /* filled_slots */
    pthread_mutex_init( _____, NULL );
}

void* producer(void* item_ptr) {
    int item = *(int*)item_ptr;
    _____ (empty_slots);
    _____ (&mutex);
    buf[in] = item;
    in = (in + 1) % BUF_SIZE;
    _____ (&mutex);
    _____ (&filled_slots);
    return NULL;
}

void* consumer(void* result_ptr) {
    _____ (&filled_slots);
    _____ (&mutex);
    int item = buf[out];
    out = (out + 1) % BUF_SIZE;
    _____ (&mutex);
    _____ (empty_slots);
    *(int*)result_ptr = item;
    return NULL;
}
```

Answer (fill in all blanks above, then explain init values below):

Q3 File I/O and File Descriptor Tables [10 Marks]

Assume the following program runs with **data.txt** (inode 2001, content: HELLO WORLD!!, 13 bytes) and **log.txt** (inode 2002, content: ABCDEFGHIJ, 10 bytes) already existing on disk. Trace the **Process Open File Table** and the **System-Wide Open File Table** after each labelled line. Use the initial state row as your template.

```

1 char wbuf[5] = "12345";
2 char rbuf[8];
3 int fdA = open("data.txt", O_RDWR);
4 int fdB = open("log.txt", O_RDWR);
5 write(fdA, wbuf, 5);
6 lseek(fdA, -3, SEEK_CUR);
7 read(fdB, rbuf, 4);
8 write(fdB, wbuf, 2);
9 if (fork() == 0) {                /* child */
10     int fdC = dup(fdA);
11     read(fdC, rbuf, 3);
12 } else {                          /* parent */
13     int fdD = open("data.txt", O_RDWR);
14     read(fdD, rbuf, 4);
15 }

```

Fill in the final state of the tables below (after all lines have executed in both parent and child). Show each process separately.

Parent Process

Child Process

System-Wide Open File Table

FD → SysFD	Sys	FD → SysFD	Sys	SysFD	Offset	RefCnt	inode Ptr
Slot 1		Slot 1					
Slot 2		Slot 2					
Slot 3		Slot 3					
Slot 4		Slot 4					
Slot 5		Slot 5					

Q4 Signals and Process Management [10 Marks]

a. Consider the following two programs. Assume all functions return without error, no signals arrive from external processes, and printf is atomic. For each possible output, write Y if it can occur, else N. [5 Marks]

Code A:

```

// Code A
int val = 10;
void handler(int sig) {
    val -= 5;
    printf("%d", val);
    exit(0);
}
int main() {
    signal(SIGUSR1, handler);
    printf("%d", val);
    pid_t c = fork();
    if (c == 0) {

```

```

        while(1);    /* spin */
    }
    kill(c, SIGUSR1);
    waitpid(-1, NULL, 0);
    val += 2;
    printf("%d", val);
    exit(0);
}

```

Code B:

```

// Code B
int x = 1;
void hA(int sig) { x = 0; }
void hB(int sig) { }
int main() {
    signal(SIGINT, hA);
    signal(SIGCONT, hB);
    pid_t c = fork();
    if (c == 0) {
        while (x == 1) pause();
        printf("p");
    } else {
        kill(c, SIGCONT);
        printf("q");
        kill(c, SIGINT);
        printf("r");
    }
}

```

Code A Outcome	Y / N?
Nothing is printed.	
"10" is printed.	
"1012" is printed.	
"1052" is printed.	
"10512" is printed.	
A process does not terminate.	

Code B Outcome	Y / N?
Nothing is printed.	
"qr" is printed.	
"qpr" is printed.	
"qrp" is printed.	
"pqr" is printed.	
A process does not terminate.	

b. Consider the following program that handles SIGALRM asynchronously. A race condition exists between the alarm handler and the main loop. Identify the race condition and show exactly where to insert `sigprocmask()` calls to guarantee that the value 500 is always printed. [3 Marks]

```

#include <signal.h>

```

```

#include <stdio.h>
volatile int count = 0;
void alarm_handler(int sig) {
    count = 0;          /* reset on alarm */
}
int main() {
    sigset_t mask;
    sigemptyset(&mask);
    sigaddset(&mask, SIGALRM);
    signal(SIGALRM, alarm_handler);
    alarm(5);
    int i;
    for (i = 0; i < 500; i++)
        count++;
    printf("count = %d\n", count);
    return 0;
}

```

Answer:

c. Explain the difference between a zombie process and an orphan process. Write a minimal C code snippet (using fork and signal) that prevents zombie processes from accumulating without calling wait() inside the parent's main loop. [2 Marks]

Answer:

Q5 Pipes, IPC, and Shell Programming [11 Marks]

a. The following program is intended to have the child send the string "ping\n" to the parent and the parent reply "pong\n" back to the child using two pipes. The program has two bugs that will cause it to deadlock or produce no output. Identify both bugs and write the corrected code. [4 Marks]

```
#include <unistd.h>
#include <stdio.h>

int main() {
    int p2c[2], c2p[2]; /* parent-to-child, child-to-parent */
    pipe(p2c); pipe(c2p);
    char buf[16];
    if (fork() == 0) {
        /* child */
        write(c2p[1], "ping\n", 5);
        read(p2c[0], buf, 5);
        printf("Child got: %s", buf);
    } else {
        /* parent */
        read(c2p[0], buf, 5);
        printf("Parent got: %s", buf);
        write(p2c[1], "pong\n", 5);
    }
    return 0;
}
```

Answer (identify bugs & corrected code):

b. A simple shell must implement the pipeline: `cmd1 | cmd2 | cmd3` where each command is an external executable. Complete the template below to implement this 3-stage pipeline. Fill every blank; do not add extra variables. [7 Marks]

```
int main() {
    int pipe1[2], pipe2[2];
    _____(pipe1);
    _____(pipe2);
    /* Stage 1: cmd1 writes to pipe1 */
    if (_____ == 0) {
        close(_____); /* close unused read end */
        dup2(_____, STDOUT_FILENO);
        close(_____);
        execlp("cmd1", "cmd1", NULL);
    }
    /* Stage 2: cmd2 reads pipe1, writes pipe2 */
    if (_____ == 0) {
        close(_____);
        dup2(_____, STDIN_FILENO);
        close(_____);
        close(_____);
        dup2(_____, STDOUT_FILENO);
        close(_____);
    }
}
```

```

        execlp("cmd2", "cmd2", NULL);
    }
    /* Stage 3: cmd3 reads pipe2, writes to terminal stdout */
    if (_____ == 0) {
        close(_____);
        dup2(_____, STDIN_FILENO);
        close(_____);
        execlp("cmd3", "cmd3", NULL);
    }
    /* Parent: close all pipe ends, wait for all children */
    close(pipe1[0]); close(pipe1[1]);
    close(pipe2[0]); close(pipe2[1]);
    _____; _____; _____;
    return 0;
}

```

Answer (fill in blanks above):

Q6 Socket Programming [10 Marks]

a. A concurrent TCP echo server must handle each client in a separate thread so that multiple clients can be served simultaneously. The partial server code below uses a single-threaded loop. There are two logic bugs. Identify them and show the corrected code. [5 Marks]

```

#define PORT 9090
#define BUFSZ 256
char buf[BUFSZ];
int main() {
    int srv = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(PORT);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    bind(srv, (struct sockaddr*)&addr, sizeof(addr));
    listen(srv, 10);
    while (1) {
        int cli = accept(srv, NULL, NULL);
        int n;
        while ((n = recv(cli, buf, BUFSZ, 0)) > 0)
            send(cli, buf, n, 0);
        close(srv); /* BUG */
    }
}

```

Answer (bugs + corrected code):

b. A server must bind and listen on all four combinations of IP addresses (192.168.10.1 and 192.168.10.2) and port numbers (8080 and 8081), each supporting at most 200 pending connections. Using the template `open_listenfd(char *host, char *port, int backlog)`, write the four `open_listenfd` calls and the definition of that helper function (using `getaddrinfo`, `socket`, `bind`, `listen`). [5 Marks]

Answer:

ANSWER KEY

Q1 — Concurrent Programming and Process Creation [16 Marks]

Code1 Answer: Possible first-printed values: 3, 4, 5, 6

Explanation: Three entities increment result (main + 2 threads); result starts at 0 and each adds 3.

Depending on interleaving:

- Only main increments before any printf → prints 3.
- Main + 1 thread increment before any printf → prints 6. Wait — result+=3 is 3+3=6. So possible values are 3, 6, 9 (all three increment before first printf). Actually:

If only ONE entity increments before the first printf → result=3 (prints 3).

If TWO entities increment before the first printf → result=6 (prints 6).

If ALL THREE increment before the first printf → result=9 (prints 9).

Race conditions also allow intermediate values: e.g., 3 or 6 or 9. **Possible: 3, 6, 9** (all three add 3 each, no partial increments since += is not atomic on all architectures — technically any value 1–9 is possible if += is non-atomic, but typical exam answer: 3, 6, 9).

Code2 Answer: First printed value is always 3

pthread_join after each create serialises execution: thread 0 completes entirely before thread 1 starts.

Thread 0 executes result+=3 (result=3) then printf → prints 3.

Code3 Answer: Possible first-printed values: 3 or 6

Mutex prevents the two worker threads from printing simultaneously, but main runs without the lock. If main increments before any thread → some first printer sees 3 or 6. 9 is impossible because at most one worker thread can be inside the mutex at a time, so two workers cannot both increment before either prints.

Code4 Answer: First printed value is always 3

fork() gives each child process its own copy of result (initially 0). Each child independently does result+=3 and prints → always 3. The parent also prints 3 for the same reason (its copy starts at 0, adds 3).

Q2 — Synchronisation [17 Marks]

2a. Problem & Fix:

Problem: Race condition — both threads can read flag==0 simultaneously, both set flag=1, both enter the critical section.

Fix: Replace the flag-based check with a mutex lock:

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
void* critical_task(void* arg) {
    pthread_mutex_lock(&lock);
    /* === critical section === */
    pthread_mutex_unlock(&lock);
    return NULL;
}
```

2b. Synchronisation Primitive Answers:

- A. none needed — reads only, no race condition.
- B. mutex — mutual exclusion for increment (read-modify-write).
- C. none needed — all threads only read; no writes.
- D. rwlock — readers acquire read lock; inserting thread acquires write lock.
- E. semaphore(3) — counting semaphore with initial value 3 (licence limit).
- F. mutex — the logger and event-poster share a queue; mutex protects enqueue/dequeue.
- G. rwlock — many threads read the inode table; VFS thread holds write lock to modify.

- H. semaphore(3) — three printers; semaphore(3) allows at most 3 threads at a time.
 I. semaphore(50) — empty_slots initialised to 50; producer calls sem_wait(empty_slots).
 J. semaphore(0) — filled_slots initialised to 0; consumer calls sem_wait(filled_slots).

2c. Producer-Consumer Fill-in Answers:

init():

```
sem_init(&empty_slots, 0, BUF_SIZE); /* 64 empty slots initially */
sem_init(&filled_slots, 0, 0); /* 0 filled slots initially */
pthread_mutex_init(&mutex, NULL);
```

producer(): sem_wait(&empty_slots); pthread_mutex_lock(&mutex); ... pthread_mutex_unlock(&mutex);
 sem_post(&filled_slots);

consumer(): sem_wait(&filled_slots); pthread_mutex_lock(&mutex); ... pthread_mutex_unlock(&mutex);
 sem_post(&empty_slots);

Q3 — File I/O and File Descriptor Tables [10 Marks]

Key trace steps:

Line 3: open("data.txt", O_RDWR): fdA→SysFD 10 (offset 0, refCnt 1, inode 2001)

Line 4: open("log.txt", O_RDWR): fdB→SysFD 11 (offset 0, refCnt 1, inode 2002)

Line 5: write(fdA, wbuf, 5): SysFD 10 offset becomes 5

Line 6: lseek(fdA, -3, SEEK_CUR): SysFD 10 offset becomes 2

Line 7: read(fdB, rbuf, 4): SysFD 11 offset becomes 4

Line 8: write(fdB, wbuf, 2): SysFD 11 offset becomes 6

Line 9: fork(): Child inherits fdA→SysFD 10 and fdB→SysFD 11; refCnt of SysFD 10 and 11 each increase to 2

Line 10 (child): dup(fdA) → fdC: fdC→SysFD 10; SysFD 10 refCnt becomes 3

Line 11 (child): read(fdC, rbuf, 3): SysFD 10 offset: 2+3=5 (shared with parent fdA)

Line 13 (parent): open("data.txt") → fdD: fdD→new SysFD 12 (offset 0, refCnt 1, inode 2001)

Line 14 (parent): read(fdD, rbuf, 4): SysFD 12 offset becomes 4

Final State:

Parent FD table: fdA→10, fdB→11, fdD→12. Child FD table: fdA→10, fdB→11, fdC→10.

System-Wide table: SysFD 10 (offset 5, refCnt 2 after child read, inode 2001); SysFD 11 (offset 6, refCnt 2, inode 2002); SysFD 12 (offset 4, refCnt 1, inode 2001).

Q4 — Signals and Process Management [10 Marks]

4a. Signal Outcome Tables:

Code A:

Nothing is printed. — N — parent always prints val (10) before fork.

"10" is printed. — Y — parent prints 10; child receives SIGUSR1 before printing; parent's waitpid returns, val+=2 never reaches printf if main exits. Actually parent does printf(val) before fork → '10' then val+=2=12 → '12'. So '1012' is possible, '10' alone requires parent to exit before printing 12.

"1012" is printed. — Y — parent prints 10, child handler prints 5, parent prints 12.

"1052" is printed. — N — not a valid execution sequence.

"10512" is printed. — N — handler executes exit(0), no further prints from child.

A process does not terminate. — N — child receives SIGUSR1 and calls exit(0); parent calls waitpid then exits.

Code B:

Nothing is printed. — N

"qr" is printed. — N — child will eventually receive SIGINT and print 'p'.

"qpr" is printed. — N — 'r' comes after second kill, so child's 'p' cannot appear between 'q' and 'r' here.

"qrp" is printed. — Y — parent prints q, sends SIGCONT (wakes child from pause), prints r, sends SIGINT; child handler sets x=0, child exits pause loop, prints p.

"pqr" is printed. — N — parent prints q before child can print p.

A process does not terminate. — N

4b. sigprocmask Answer:

Race condition: SIGALRM can fire between the loop's increment and the final printf, resetting count to 0.
Fix: block SIGALRM before the loop and unblock after.

```
sigprocmask(SIG_BLOCK, &mask;, NULL); /* add before loop */
for (i = 0; i < 500; i++) count++;
sigprocmask(SIG_UNBLOCK, &mask;, NULL); /* add before printf */
printf("count = %d\n", count);
```

4c. Zombie vs Orphan & Prevention:

Zombie: A process that has finished but whose exit status has not yet been reaped by wait(). Its PCB remains in the process table.

Orphan: A process whose parent has terminated before it. It is reparented to init (PID 1), which reaps it automatically.

Preventing zombies without blocking wait() in main loop — use SIGCHLD:

```
void sigchld_handler(int sig) {
    while (waitpid(-1, NULL, WNOHANG) > 0);
}
/* In main: */
signal(SIGCHLD, sigchld_handler);
```

Q5 — Pipes, IPC, and Shell Programming [11 Marks]

5a. Two Bugs in Pipe Program:

Bug 1: Neither process closes the write ends it does not use. Because the parent holds an open write end to c2p[1], read(c2p[0]) in the parent will never see EOF — but more critically the child's write won't unblock the parent until the unused ends are closed. Fix: each side must close the ends it doesn't use.

Bug 2: The parent writes pong only *after* it has read from c2p. But the child writes to c2p first and then immediately tries to read from p2c. This is fine sequentially here, but the parent doesn't read until child has written — the real bug is that neither process closes unused pipe ends, causing blocking.

Corrected skeleton:

```
if (fork() == 0) {
    close(c2p[0]); close(p2c[1]); /* close unused ends in child */
    write(c2p[1], "ping\n", 5);
    read(p2c[0], buf, 5);
    printf("Child got: %s", buf);
} else {
    close(c2p[1]); close(p2c[0]); /* close unused ends in parent */
    read(c2p[0], buf, 5);
    printf("Parent got: %s", buf);
    write(p2c[1], "pong\n", 5);
}
```

5b. Three-Stage Pipeline Filled Blanks:

```
pipe(pipe1); pipe(pipe2);
/* Stage 1 */ if (fork() == 0) {
    close(pipe1[0]);
    dup2(pipe1[1], STDOUT_FILENO); close(pipe1[1]);
```

```

        execlp("cmd1", "cmd1", NULL); }
/* Stage 2 */ if (fork() == 0) {
    close(pipe1[1]);
    dup2(pipe1[0], STDIN_FILENO); close(pipe1[0]);
    close(pipe2[0]);
    dup2(pipe2[1], STDOUT_FILENO); close(pipe2[1]);
    execlp("cmd2", "cmd2", NULL); }
/* Stage 3 */ if (fork() == 0) {
    close(pipe2[1]);
    dup2(pipe2[0], STDIN_FILENO); close(pipe2[0]);
    execlp("cmd3", "cmd3", NULL); }
/* Parent */
wait(NULL); wait(NULL); wait(NULL);

```

Q6 — Socket Programming [10 Marks]

6a. Two Logic Bugs:

Bug 1: `close(srv)` inside the loop closes the *server* socket (the listening socket) instead of the *client* socket after each session ends. This terminates the server's ability to accept new connections. Fix: change to `close(cli)`;

Bug 2: The server handles each client in the main loop sequentially — new connections are blocked until the current client finishes. Fix: after `accept()`, spawn a thread to handle the client:

```

int cli = accept(srv, NULL, NULL);
pthread_t tid;
int *pCli = malloc(sizeof(int));
*pCli = cli;
pthread_create(&tid, NULL, handle_client, pCli);
pthread_detach(tid);

```

6b. `open_listenfd` helper + 4 calls:

```

int open_listenfd(char *host, char *port, int backlog) {
    struct addrinfo hints = {0}, *res;
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = AI_PASSIVE;
    getaddrinfo(host, port, &hints, &res);
    int fd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
    int opt = 1;
    setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    bind(fd, res->ai_addr, res->ai_addrlen);
    listen(fd, backlog);
    freeaddrinfo(res);
    return fd;
}

int s1 = open_listenfd("192.168.10.1", "8080", 200);
int s2 = open_listenfd("192.168.10.1", "8081", 200);
int s3 = open_listenfd("192.168.10.2", "8080", 200);
int s4 = open_listenfd("192.168.10.2", "8081", 200);

```
